

Building Intelligent Observability and Filtering Infrastructure for BOOM

1. Background

Astronomical time-domain surveys, such as those conducted by LIGO and ZTF, generate a deluge of transient event data. These events, often originating from extreme phenomena like black hole mergers or neutron star collisions, must be filtered, classified, and followed up in real time. To support this workflow, the Burst & Outburst Observations Monitor (BOOM) project at the University of Minnesota serves as a real time alert broker. It collects gravitational wave candidate alerts and redistributes refined information to astronomers and follow up facilities. However, as BOOM scales up, two technical bottlenecks have emerged: (1) a lack of comprehensive monitoring tools that track the system's internal performance in real time, and (2) a filtering pipeline that could benefit from enhanced classification logic to reduce false positives and highlight high-priority events.

Reliable infrastructure is critical to ensure high uptime and responsiveness in these pipelines. BOOM currently uses Kafka, Redis, MongoDB, and APIs built using Rust and Python. Tools like the Rust tracing crate allow for structured logging and diagnostics across async workflows but remain under utilized. Meanwhile, the filtering logic relies on parameters like chirp mass and false alarm rate (FAR) but has not yet been optimized to target scientifically valuable candidates (e.g., events with chirp mass > 22). This proposal aims to address both challenges, ultimately enhancing the reliability and scientific utility of BOOM.

2. Research Question

Our goal is to answer the question: Can we improve the observability and scientific precision of the BOOM pipeline by implementing structured monitoring tools and optimizing its filtering logic for high-value gravitational-wave events?

3. Methodology

To address the research question, the project will be divided into two core phases: monitoring infrastructure and filtering optimization.

- a. Monitoring Infrastructure (Rust + tracing) - The core of this project focuses on building a structured, extensible logging system for BOOM's API. We will replace basic Actix middleware logging with a richer implementation using the tracing and tracing-actix-web crates. This will allow us to:
 - Automatically captured detailed logs for each API request, including IP address, endpoint accessed, latency, and request headers.

- Attach spans to async functions using ‘#[instrument]’ macros, which provide contextual information like function arguments and lifetimes.
- Enable error tracing using custom macros (log_err!, as_err!) that emit consistent error logs at precise call sites, even when the same error can be triggered in multiple places.
- Capture structured events during key lifecycle points: API start-up/shutdown, alert ingestion, filter processing, and Redis/Kafka interactions.
- Configure filters dynamically via environment variables (RUST_LOG, RUST_LOG_SPAN_EVENTS) to reduce console clutter or turn on span tracking on-demand.

The logging system will also be structured to account for the multi-catalog design used in BOOM, where each survey (e.g., ZTF, LSST) maintains separate databases for individual candidates (alerts), their associated image cutouts, and astronomical object records. By tagging spans and events to these stages, we can trace how data moves through the BOOM system and identify points of failure or delay with precision.

This system will significantly improve debugging, observability, and system transparency. If performance overhead from console logging becomes significant, we will migrate to asynchronous logging using tracing-appender for file-based persistence.

- b. Filtering and Classification Optimization - Parallel to improving logging, the project will evaluate the filtering logic BOOM uses to decide whether a gravitational wave candidate is worth forwarding. The steps are reviewing filtering thresholds (e.g., chirp mass, FAR, distance) and how they're implemented, replaying a set of historical alerts from GraceDB and injecting synthetic test data, testing classification accuracy for known true/false positives, proposing better prioritization rules, particularly around high-value sources (e.g., mchirp) or tighter FAR constraints.

The updated filter rules will be tested for both latency (how fast alerts move through the system) and quality (how relevant the filtered alerts are).

- c. Tools and Data

- Programming Languages: Rust, Python
- Technologies: Kafka, Redis, Docker, tracing, MongoDB
- Data Sources: GraceDB public alert stream, simulated alert set.

- d. Contingency Plan - If real-time metrics prove difficult to visualize via Grafana, fallback to terminal-based reporting. If replaying alerts leads to malformed data, test on pre-cleaned samples or reduce input scope.

4. Timeline (120 Hours)

1-2 weeks: Integrate tracing into the BOOM API, log requests/responses and error events.

3-4 weeks: Add spans to key async functions, test span depth and field tracking.

5-6 weeks: Analyze and replay existing filters, benchmark classification accuracy.

7-8 weeks: Redesign filter rules based on test data, validate with mock alerts.

9-10 weeks: Finalize documentation, prepare log samples, and complete UROP report & presentation.

5. Impact

This project aims to make BOOM more robust, reliable, and scientifically productive. A fully featured logging system will simplify debugging and long-term monitoring while enabling fine-grained analysis of system behavior. Improvements to the filtering logic will increase the quality of alerts forwarded to astronomers, reducing wasted follow-ups and improving real-time discovery. The tools and knowledge gained from this work will benefit future backend development efforts in astronomical software and give the researcher real-world experience in Rust observability, async programming, and scientific pipeline design.

6. References

- Singer, L.P. & Price, L.R. (2016). Rapid Bayesian position reconstruction for gravitational-wave transients. <https://arxiv.org/abs/1508.03634>
- Abbott, B.P. et al. (2016). Observation of Gravitational Waves from a Binary Black Hole Merger. Phys. Rev. Lett. <https://journals.aps.org/prl/abstract/10.1103/PhysRevLett.116.061102>
- Chaudhary, S.S. et al. (2023). Low-latency gravitational wave alert products and their performance in anticipation of the fourth LIGO Virgo KAGRA run. <https://arxiv.org/abs/2308.04545>
- Tokio.rs. (2023). `tracing` crate documentation. <https://docs.rs/tracing>
- GraceDB: Gravitational-Wave Candidate Event Database. <https://gracedb.ligo.org>
- tracing-actix-web. (2023). Structured logging middleware for Actix Web. <https://docs.rs/tracing-actix-web>